Lecture 9

Algorithm Design Techniques

Algorithm Design Techniques

A Comprehensive Guide for CS Students

Seven Paradigms: Brute Force · Recursive · Divide Only · Divide & Conquer
Greedy · Dynamic Programming · Backtracking

© 2026 Ahmed Elrefaiy

Algorithm Design Techniques

A Comprehensive Guide for CS Students

Seven Paradigms: Brute Force · Recursive · Divide Only · Divide & Conquer
Greedy · Dynamic Programming · Backtracking

1. Introduction

Algorithm design is the art of choosing the right problem-solving strategy. Each paradigm represents a different **way of thinking** about a problem — not just a coding style, but a mental model.

#	Paradigm	Core Idea
1	Brute Force	Exhaustively try every possible solution
2	Recursive	Break the problem into the same problem at a smaller scale
3	Divide Only	Split to narrow the search space — process only one side
4	Divide & Conquer	Split → Solve each half recursively → Merge results
5	Greedy	Pick the locally best option at each step, never look back
6	Dynamic Prog.	Cache results of overlapping subproblems to avoid recomputation
7	Backtracking	Explore all paths, abandon dead ends early (prune)

2. The Seven Algorithm Design Paradigms

2.1 Brute Force

Brute Force is the most straightforward approach: **try every possible solution** and pick the one that works. No creativity needed — but guarantees correctness if given enough time.

How It Thinks

Generate all candidates → Test each one → Return the first (or best) valid answer.

When to Use

- The problem size is very small (n ≤ 20 or so)
- No known efficient algorithm exists yet
- You need a correct baseline to verify a smarter algorithm
- The problem has no exploitable structure

Complexity

Dimension	Typical
Time	O(2■), O(n!), O(n■) — exponential or worse
Space	O(1) — no extra structures needed

Classic Algorithm Examples

- **Linear Search** — scan every element to find a target
- **Bubble Sort** — compare every adjacent pair repeatedly until sorted
- **Selection Sort** — find the minimum in the unsorted part each pass
- **Naive String Matching** — try matching the pattern at every position in the text
- **Permutation Generation** — enumerate all n! orderings to find the best one

■ Recognition Clues — How to Spot This Paradigm

```
❖ "find ALL solutions / combinations / subsets"
❖ "check every possible ..."
❖ Small constraint: n ≤ 15 or n ≤ 20
❖ No sorting, no structure mentioned
❖ "generate all permutations / arrangements"
```

```
◆ Password cracking, exhaustive search
```

Ask yourself: *Is there any smarter pattern here, or must I try everything? If 'try everything' and n is small → Brute Force.*

2.2 Recursive

A Recursive algorithm solves a problem by **calling itself on a smaller version** of the same problem. It is a *thinking style* — many paradigms (D&C;, DP, Backtracking) are built on top of recursion.

Steps

1. **Base Case** — The simplest input that can be answered directly
2. **Recursive Case** — Break the problem into a smaller instance and call yourself
3. **Return** — Combine or pass back the result up the call stack

When to Use

- The problem is naturally self-similar (same structure at every scale)
- Tree or graph traversal (every node leads to more nodes)
- Mathematical definitions are recursive (Fibonacci, factorial, GCD)
- Combinatorial generation (subsets, permutations)

Complexity

Dimension	Typical
Time	Depends on branching factor and depth — analyze with a recurrence relation
Space	O(depth) for the call stack

Classic Algorithm Examples

- **Factorial n!** — factorial(n) = n × factorial(n-1)
- **Tree Traversals** — In-order, Pre-order, Post-order
- **Tower of Hanoi** — move n disks by recursively moving n-1
- **Flood Fill** — paint connected pixels by recursing into neighbors

■ Recognition Clues — How to Spot This Paradigm

```
◆ The problem has a 'smaller version of itself' inside
◆ Tree / graph structure mentioned (nodes, children, parent)
```

```
◆ 'Define f(n) in terms of f(n-1)' type wording
◆ Words: hierarchical, nested, branching
◆ Mathematical sequences with recurrence relations
```

Ask yourself: *Can I define the answer to this problem using the answer to a smaller version of the same problem? If yes → Recursion.*

2.3 Divide Only (Without Conquer)

Sometimes you only need to **split data into partitions** — there is no merging or recursive solving of both halves. The answer lives in **one** partition. You do *not* combine results from both sides.

How It Thinks

Divide the input → Decide which part has the answer → Search only that part → Repeat.

When to Use

- The problem has a monotone or sorted structure
- You can eliminate half (or more) of the search space each step
- Partitioning is the goal itself

Complexity

Dimension	Typical
Time	O(log n) — halving the search space each step
Space	O(1) iterative, O(log n) recursive

Classic Algorithm Examples

- **Binary Search** — compare to midpoint → go left or right. Only ONE half is ever explored
- **QuickSelect** — find the k-th smallest — partition, then recurse into ONE side
- **Ternary Search** — divide into thirds, eliminate one third each time

■ Recognition Clues — How to Spot This Paradigm

```
◆ Array/data is SORTED (or can be sorted)
◆ 'Find the position of X' in a sorted structure
◆ 'Find the first/last element satisfying condition Y'
```

```
◆ The answer space is monotone
◆ O(log n) is expected/required
◆ Words: search, locate, find index, lower bound, upper bound
```

Ask yourself: *After I check the midpoint, can I ELIMINATE half the remaining search space? If yes → Divide Only (Binary Search family).*

2.4 Divide and Conquer

Divide & Conquer is the **full three-step cycle**: split the problem into independent subproblems, solve **both** halves recursively, then **merge** the results into a final answer.

Steps

1. **Divide** — Break the input into smaller, independent subproblems
2. **Conquer** — Solve each subproblem recursively
3. **Combine** — Merge the subproblem solutions into the overall answer

When to Use

- Subproblems are independent (no shared state)
- The problem can be split into roughly equal halves
- There is a clear merge step to combine results
- Parallelism is possible (each half can run concurrently)

Complexity

Dimension	Typical
Time	O(n log n) typically — via Master Theorem T(n) = 2T(n/2) + O(n)
Space	O(log n) — recursion stack depth

Classic Algorithm Examples

- **Merge Sort** — split → sort both halves → merge in O(n) → O(n log n)
- **Quick Sort** — partition around pivot → sort both sides
- **Closest Pair of Points** — split plane, solve each half, check strip
- **Strassen Matrix Multiply** — 7 subproblems instead of 8
- **FFT** — split into even/odd indices recursively → O(n log n)

■ Recognition Clues — How to Spot This Paradigm

```
◆ 'Sort the array' → almost always D&C;
◆ Problem can be split into two INDEPENDENT halves
◆ There is a natural 'merge' or 'combine' step
◆ T(n) = 2T(n/2) + O(n) pattern is visible
◆ Words: sort, merge, closest, majority element, inversion count
◆ Parallelizable: each half can be solved independently
```

Ask yourself: *Can I split this into independent halves, solve each, then combine? If yes AND both halves must be processed → Divide & Conquer.*

2.5 Greedy Algorithms

A Greedy algorithm builds the solution **one step at a time**, always choosing the option that looks best *right now* — without considering future consequences. It **never revises** past decisions.

Steps

1. **Greedy Choice Property** — a global optimum can be reached by locally optimal choices
2. **Optimal Substructure** — an optimal solution to the whole contains optimal solutions to subproblems

When to Use

- Both required properties (below) can be formally proven
- Scheduling, interval selection, or resource allocation problems
- Graph problems: MST, shortest paths with non-negative weights
- When O(n log n) or O(n) is required and correctness can be shown

Complexity

Dimension	Typical
Time	O(n log n) or O(n) — dominated by sorting step
Space	O(1) to O(n)

Classic Algorithm Examples

- **Activity Selection** — pick the activity finishing earliest → O(n log n)

- **Huffman Coding** — always merge the two smallest frequency nodes first
- **Kruskal's MST** — always add the cheapest edge that doesn't form a cycle
- **Prim's MST** — always extend the tree via the cheapest available edge
- **Dijkstra's SSSP** — always relax the nearest unvisited vertex
- **Fractional Knapsack** — take items by highest value-per-weight ratio

■ Recognition Clues — How to Spot This Paradigm

```
◆ 'maximize' or 'minimize' something with a single pass
◆ 'sort by X, then pick best' pattern
◆ Scheduling: finish the most tasks, earliest deadline first
◆ No need to reconsider or undo choices
◆ Items have a ratio (value/weight, profit/time) → sort by ratio
◆ Words: MST, shortest path, activity selection, interval, huffman
✗ Items CANNOT be split (0/1 decision) → Greedy likely fails
✗ You find a counterexample where local-best ≠ global-best
```

Ask yourself: *If I always pick the best available option right now, will I always reach the best total result? If provably yes → Greedy. If unsure → try DP instead.*

2.6 Dynamic Programming (DP)

DP solves problems with **overlapping subproblems**. Instead of recomputing the same subproblem multiple times, DP **stores (memoizes) the result** the first time and looks it up on future calls.

Steps

1. **Optimal Substructure** — the optimal solution is built from optimal sub-solutions
2. **Overlapping Subproblems** — the same subproblems are encountered again and again

When to Use

- Both required properties hold (see below)
- Recursive solution has exponential time due to repeated subproblems
- Optimization problems: minimize/maximize over a sequence/partition
- Counting problems: 'how many ways to ...'

Complexity

Dimension	Typical
Time	$O(n^2)$, $O(nW)$, $O(nm)$ — polynomial
Space	$O(n)$ to $O(nW)$ for the table; sometimes reducible

Classic Algorithm Examples

- **Fibonacci (Memoized)** — cache results → $O(n)$ vs $O(2^n)$ brute force
- **0/1 Knapsack** — dp[i][w] = max value using first i items with capacity w
- **Longest Common Subsequence** — dp[i][j] = LCS of first i and j chars
- **Edit Distance** — dp[i][j] = min edits to convert prefix i to prefix j
- **Coin Change** — dp[amount] = min coins to make that amount
- **Matrix Chain Multiplication** — dp[i][j] = min cost to multiply matrices i..j

■ Recognition Clues — How to Spot This Paradigm

```
◆ 'Minimum / maximum number of ways to reach X'
◆ 'Count the number of ways to ...'
◆ Recursive solution has exponential time + same subproblems repeat
◆ 'Optimal' + 'subsequence / substring / path / partition'
◆ Words: knapsack, LCS, edit distance, coin change, staircase, palindrome
◆ Drawing the recursion tree shows repeated subtrees
■ Can I define a state dp[i] or dp[i][j] for the subproblem?
■ Can I write a recurrence: dp[i] = f(dp[i-1], ...)?
■ Do I have clear base cases?
```

Ask yourself: *Am I solving the same smaller problem multiple times in my recursion? If yes → Dynamic Programming.*

2.7 Backtracking

Backtracking is a **smarter version of Brute Force**. It explores all possible solutions via recursion, but **prunes branches early** — the moment a partial solution cannot possibly lead to a valid complete solution, it abandons that path.

Steps

1. **Choose** — Pick an option for the current decision point
2. **Explore** — Recurse into the next decision point

3. **Unchoose** — If the path failed, undo the choice and try the next option

When to Use

- Constraint satisfaction problems (N-Queens, Sudoku, Map Coloring)
- Combinatorial search: all subsets, all permutations, all paths
- Puzzle solving where the search space is too large for brute force
- When partial solutions can be validated before completion

Complexity

Dimension	Typical
Time	Worst case O(2■) or O(n!), but pruning reduces practical time dramatically
Space	O(depth of recursion tree) for the call stack

Classic Algorithm Examples

- **N-Queens** — place n queens on an n×n board with no conflicts
- **Sudoku Solver** — fill cells, backtrack when a row/col/box has conflict
- **Subset Sum** — find all subsets that sum to a target value
- **Graph Coloring** — color vertices so no adjacent vertices share a color
- **Hamiltonian Path** — find a path that visits every vertex exactly once
- **Word Search** — find a word in a 2D grid using DFS + backtracking

■ Recognition Clues — How to Spot This Paradigm

```
◆ 'Find ALL solutions / arrangements satisfying constraints'
◆ 'Is there a valid assignment / coloring / placement?'
◆ Decision at each step + multiple options + constraints
◆ You must UNDO a decision if it leads to failure
◆ Words: N-Queens, Sudoku, maze, word search, graph coloring
◆ Brute force would work but is too slow → pruning makes it feasible
◆ The search space is a tree (each node = partial solution)
```

Ask yourself: *Do I need to try multiple options, with the ability to undo and try another if one fails? If yes → Backtracking.*

---

3. Paradigm Comparison at a Glance

Paradigm	Time	Space	Best For	Key Weakness
Brute Force	O(2■)–O(n!)	O(1)	Small n, baseline	Extremely slow for large n
Recursive	Varies	O(depth)	Self-similar problems	Stack overflow; may recompute
Divide Only	O(log n)	O(1)	Sorted / monotone data	Requires sorted structure
Divide & Conquer	O(n log n)	O(log n)	Independent subproblems	Merge cost can dominate
Greedy	O(n log n)	O(1)–O(n)	Local-optimal → global	Wrong without formal proof
Dynamic Prog.	$O(n^2)$–O(nW)	O(n)–O(nW)	Overlapping subproblems	High memory for large tables
Backtracking	O(2■) pruned	O(depth)	Constraint satisfaction	Slow without good pruning

4. Problem 1 — Finding Maximum in an Array

Problem: Given an array `arr` of `n` integers, find and return the maximum element.

Input: arr = [3, 7, 1, 9, 4, 6] → **Output:** 9

4.1 Brute Force

Compare every pair of elements. For each element, check if any other element beats it.

Algorithm

```
FindMax_BruteForce(arr, n):
FOR i = 0 TO n-1:
isMax = TRUE
FOR j = 0 TO n-1:
IF arr[j] > arr[i]:
isMax = FALSE
BREAK
IF isMax:
RETURN arr[i]
```

Time	Space	Notes
$O(n^2)$	O(1)	Two nested loops — artificially inefficient for this problem

4.2 Recursive

The maximum of an array is either the last element, or the max of all elements before it.

Algorithm

```
FindMax_Recursive(arr, n):
IF n == 1: // Base case
RETURN arr[0]
maxOfRest = FindMax_Recursive(arr, n-1)
IF arr[n-1] > maxOfRest:
RETURN arr[n-1]
ELSE:
```

```
RETURN maxOfRest
```

Time	Space	Notes
O(n)	O(n)	Linear recursion depth — elegant and natural

4.3 Divide Only (Linear Scan)

March forward through the array, narrowing the 'unsolved' portion by one each step.

Algorithm

```
FindMax_LinearScan(arr, n):
max = arr[0]
i = 1

WHILE i < n:
IF arr[i] > max:
max = arr[i]
i = i + 1

RETURN max
```

Time	Space	Notes
O(n)	O(1)	True Divide Only cannot skip half — max could be anywhere

4.4 Divide and Conquer

Split the array in half. Find the max of each half recursively. Overall max = larger of the two.

Algorithm

```
FindMax_DC(arr, left, right):
IF left == right: // Base case
RETURN arr[left]

mid = (left + right) / 2

maxLeft = FindMax_DC(arr, left, mid)
maxRight = FindMax_DC(arr, mid + 1, right)

IF maxLeft > maxRight: RETURN maxLeft
```

```
ELSE: RETURN maxRight
```

Time	Space	Notes
O(n)	O(log n)	T(n)=2T(n/2)+O(1) — visits all n elements; benefit is parallelism

4.5 Greedy

Greedily maintain the best element seen so far. If current element is better, update it.

Algorithm

```
FindMax_Greedy(arr, n):
max = arr[0] // Greedy initial choice
FOR i = 1 TO n-1:
IF arr[i] > max:
max = arr[i] // Greedy update — never reconsider
RETURN max
```

Time	Space	Notes
O(n)	O(1)	■ The standard algorithm — greedy choice property holds

4.6 Dynamic Programming

Define dp[i] = maximum value in arr[0..i]. Compute bottom-up. Answer is dp[n-1].

Algorithm

```
FindMax_DP(arr, n):
dp[0] = arr[0] // Base case
FOR i = 1 TO n-1:
IF arr[i] > dp[i-1]:
dp[i] = arr[i]
ELSE:
dp[i] = dp[i-1]
RETURN dp[n-1]
```

Time	Space	Notes
O(n)	O(n)	■■ Overkill — no overlapping subproblems; shown for illustration only

4.7 Backtracking

No constraint to prune on — degenerates to simple recursion here. Not a natural fit.

Algorithm

```
FindMax_Backtrack(arr, n, index, currentMax):
IF index == n:
RETURN currentMax
// No pruning possible — max could be anywhere
newMax = MAX(currentMax, arr[index])
RETURN FindMax_Backtrack(arr, n, index+1, newMax)
// Initial call:
FindMax_Backtrack(arr, n, 0, -INFINITY)
```

Time	Space	Notes
O(n)	O(n)	■■ No constraints to prune — adds no benefit over linear scan

Summary — All Approaches for Array Maximum

Approach	Time	Space	Natural Fit?
Brute Force	$O(n^2)$	O(1)	■ Artificially inefficient
Recursive	O(n)	O(n)	■ Elegant self-similar call
Divide Only	O(n)	O(1)	■■ Partial — linear scan
Divide & Conquer	O(n)	O(log n)	■ Good template demonstration
Greedy	O(n)	O(1)	■ The standard algorithm
Dyn. Programming	O(n)	O(n)	■ Overkill — no overlapping subproblems
Backtracking	O(n)	O(n)	■ No constraints to prune

5. Problem 2 — Fibonacci Numbers

Problem: Compute the n-th Fibonacci number.

F(0) = 0, F(1) = 1, F(n) = F(n-1) + F(n-2) for n ≥ 2

Examples: F(5) = 5, F(7) = 13, F(10) = 55

5.1 Brute Force — Naive Recursion

Directly implement the recursive definition with no optimization. Recompute everything from scratch.

Algorithm

```
Fib_BruteForce(n):
IF n == 0: RETURN 0
IF n == 1: RETURN 1
// Brute force: recompute both subproblems completely
RETURN Fib_BruteForce(n-1) + Fib_BruteForce(n-2)
```

Time	Space	Notes
O(2■)	O(n)	F(50) requires ~10¹■ operations — completely impractical

5.2 Recursive — Call Tree (The Problem)

The pure recursive form IS brute force for Fibonacci. The problem is massive repeated computation:

```
Fib(5)
/ \
Fib(4) Fib(3)
/ \ / \
Fib(3) Fib(2) Fib(2) Fib(1)
/ \
Fib(2) Fib(1)
Fib(3) computed 2 times
Fib(2) computed 3 times <- wasted work
Fib(1) computed 5 times <- wasted work
```

The solution to this waste is **memoization** — addressed in Section 5.6 (Dynamic Programming).

5.3 Divide Only — Iterative

Walk forward from base cases, using only the previous two values. No merging needed.

Algorithm

```
Fib_Iterative(n):
IF n == 0: RETURN 0
IF n == 1: RETURN 1

prev2 = 0 // F(i-2)
prev1 = 1 // F(i-1)

FOR i = 2 TO n:
current = prev1 + prev2
prev2 = prev1
prev1 = current

RETURN prev1
```

Time	Space	Notes
O(n)	O(1)	■ Most practical solution for everyday use

5.4 Divide and Conquer — Matrix Exponentiation

F(n) is the top-left element of matrix [[1,1],[1,0]] raised to the power n. Matrix exponentiation uses D&C; to compute the n-th power in O(log n).

```
MatrixMultiply(A, B): // 2x2 matrices
C[0][0] = A[0][0]*B[0][0] + A[0][1]*B[1][0]
C[0][1] = A[0][0]*B[0][1] + A[0][1]*B[1][1]
C[1][0] = A[1][0]*B[0][0] + A[1][1]*B[1][0]
C[1][1] = A[1][0]*B[0][1] + A[1][1]*B[1][1]
RETURN C

MatrixPower(M, n): // Divide & Conquer on exponent
IF n == 1: RETURN M
IF n is EVEN:
half = MatrixPower(M, n/2)
RETURN MatrixMultiply(half, half)
```

```
ELSE:
RETURN MatrixMultiply(M, MatrixPower(M, n-1))

Fib_Matrix(n):
IF n == 0: RETURN 0
M = [[1,1],[1,0]]
result = MatrixPower(M, n)
RETURN result[0][1]
```

Time	Space	Notes
O(log n)	O(log n)	■ Best time complexity — used for very large n

5.5 Greedy — Not Applicable

Fibonacci **does not have a greedy solution**. There is no 'locally optimal choice' at each step — computing F(n) requires both F(n-1) and F(n-2). No greedy choice property exists here.

5.6 Dynamic Programming — The Ideal Solution

DP is the **ideal paradigm** for Fibonacci. Each value is computed exactly once.

Top-Down: Memoization

```
memo = array of size n+1, initialized to -1

Fib_Memo(n):
IF n == 0: RETURN 0
IF n == 1: RETURN 1
IF memo[n] != -1:
RETURN memo[n] // Cache hit — O(1) lookup
memo[n] = Fib_Memo(n-1) + Fib_Memo(n-2)
RETURN memo[n]
```

Bottom-Up: Tabulation

```
Fib_Tabulation(n):
dp[0] = 0
dp[1] = 1
FOR i = 2 TO n:
```

```
dp[i] = dp[i-1] + dp[i-2]
RETURN dp[n]
```

Space-Optimized — O(1) Space

```
Fib_DP_Optimized(n):
prev2 = 0
prev1 = 1
FOR i = 2 TO n:
current = prev1 + prev2
prev2 = prev1
prev1 = current
RETURN prev1
```

Variant	Time	Space
Memoization	O(n)	O(n)
Tabulation	O(n)	O(n)
Optimized	O(n)	O(1)

5.7 Backtracking — Not Applicable

Fibonacci has **no constraint structure to prune on**. Without pruning conditions, backtracking is identical to brute force recursion. Use DP or iterative instead.

Summary — All Approaches for Fibonacci

Approach	Time	Space	Practical?
Brute Force (recursive)	O(2■)	O(n)	■ Too slow for n > 40
Divide Only (iterative)	O(n)	O(1)	■ Simple and efficient
Divide & Conquer (matrix)	O(log n)	O(log n)	■ Best for very large n
Greedy	N/A	N/A	■ Not applicable
DP (memoization)	O(n)	O(n)	■ Intuitive, educational
DP (tabulation)	O(n)	O(n)	■ Systematic, no stack overflow
DP (optimized)	O(n)	O(1)	■ Best practical solution

Approach	Time	Space	Practical?
Backtracking	O(2■)	O(n)	■ Same as brute force

6. The Knapsack Problem

Problem: You have a knapsack with weight capacity **W** and **n** items, each with weight w[i] and value v[i]. Choose items to maximize total value without exceeding capacity W.

	Fractional Knapsack	0/1 Knapsack
Items splittable?	■ Yes	■ No
Correct algorithm	Greedy	Dynamic Programming
Time complexity	O(n log n)	O(n × W)
Greedy correct?	■ Yes — proven	■ No — fails

6.1 Fractional Knapsack — Greedy Works! ■

Items can be split. Taking the highest value-per-weight ratio item is always optimal.

Algorithm

```
FractionalKnapsack(items, W):
FOR each item i:
ratio[i] = value[i] / weight[i]

SORT items BY ratio DESCENDING

totalValue = 0
remainingCap = W

FOR each item i in sorted order:
IF weight[i] <= remainingCap:
TAKE entire item i
totalValue = totalValue + value[i]
remainingCap = remainingCap - weight[i]
ELSE:
fraction = remainingCap / weight[i]
totalValue = totalValue + fraction * value[i]
remainingCap = 0
BREAK

RETURN totalValue
```

Worked Example — Capacity W = 15 kg

Item	Weight	Value	Ratio ($/kg)
A	10 kg	$60	6.0
B	5 kg	$50	10.0
C	7 kg	$70	10.0

Steps: Sort: B(10.0), C(10.0), A(6.0) → Take all B: 5kg/$50 → Take all C: 12kg/$120 → Take 3/10 of A: 15kg/**$138**

Result: $138 — Greedy gave the exact optimal answer ■

6.2 0/1 Knapsack — Greedy FAILS, DP Needed ■

Each item: take (1) or leave (0). No fractions. **Greedy fails** because the best-ratio item might block a better combination.

Counter-Example — Capacity W = 50 kg

Item	Weight	Value	Ratio
1	10 kg	$60	6.0
2	20 kg	$100	5.0
3	30 kg	$120	4.0

Greedy picks: Items {1,2} → value = **$160** | **Optimal:** Items {2,3} → value = **$220 ■ (Greedy missed $60!)**

DP Solution — Recurrence

```
// If we DON'T take item i:
dp[i][w] = dp[i-1][w]

// If we DO take item i (only if weight[i] <= w):
dp[i][w] = MAX(dp[i-1][w], value[i] + dp[i-1][w - weight[i]])
```

Algorithm — Bottom-Up Tabulation

```
Knapsack_01(items, n, W):
```

```
// Initialize: dp[0..n][0..W] = 0
FOR i = 0 TO n: FOR w = 0 TO W: dp[i][w] = 0

FOR i = 1 TO n:
FOR w = 0 TO W:
dp[i][w] = dp[i-1][w] // Don't take item i
IF weight[i] <= w:
takeValue = value[i] + dp[i-1][w - weight[i]]
IF takeValue > dp[i][w]:
dp[i][w] = takeValue // Take item i

RETURN dp[n][W]
```

DP Table Trace (W = 50)

dp[i][w]	w=0	w=10	w=20	w=30	w=40	w=50
i=0 (no items)	0	0	0	0	0	0
i=1 (w=10, v=$60)	0	60	60	60	60	60
i=2 (w=20, v=$100)	0	60	100	160	160	160
i=3 (w=30, v=$120)	0	60	100	160	180	220

dp[3][50] = 220 → Optimal! DP found the answer that Greedy missed. ■

Space Optimization — O(W) Space

```
Knapsack_01_Optimized(items, n, W):
dp[0..W] = 0

FOR i = 1 TO n:
// Traverse RIGHT TO LEFT to avoid counting item i twice
FOR w = W DOWNTO weight[i]:
IF value[i] + dp[w - weight[i]] > dp[w]:
dp[w] = value[i] + dp[w - weight[i]]

RETURN dp[W]
```

7. Conclusion — Choosing the Right Paradigm

Decision Flowchart

```
Is n very small (<= 20)?
YES --> Brute Force or Backtracking

Does the problem have self-similar structure?
YES --> Try Recursion first
Does it have overlapping subproblems?
YES --> Add memoization --> Dynamic Programming
Are subproblems independent and BOTH sides needed?
YES --> Divide and Conquer
Can you eliminate half the search space each step?
YES --> Divide Only (Binary Search family)

Can you prove local-optimal = global-optimal?
YES --> Greedy

Does the problem have constraints? (valid/invalid partial solutions)
YES --> Backtracking (with pruning)

Does it have overlapping subproblems + optimal substructure?
YES --> Dynamic Programming
```

Recognition Clues — Master Summary

Paradigm	Key Trigger Phrases	Typical Keywords
Brute Force	'try all', small n <= 20, no structure	enumerate, exhaustive, all combinations
Recursive	self-similar definition, trees, graphs	node, children, subtree, depth, height
Divide Only	sorted array, eliminate half	binary search, lower bound, find index
Divide & Conquer	sort, merge, both halves needed	merge sort, closest pair, inversions
Greedy	maximize/minimize + sort + local best	scheduling, interval, MST, ratio, earliest
Dynamic Prog.	optimize + overlapping + count ways	knapsack, LCS, edit distance, coin change
Backtracking	all solutions + constraints + undo	N-Queens, Sudoku, subsets, permutations

"Understanding these paradigms is not about memorizing code.
It is about developing the instinct to recognize which thinking pattern a problem calls for."